PHP Database Masterclass

Comprehensive Tutorial on Connections, Security, and CRUD

VERSION 2024.1

WHY DATABASES MATTER?

In modern web development, PHP acts as the "brain" while the database acts as the "memory." Without a connection, your web app is static.

- **Persistence:** Store user info, posts, and products.
- **User Profiles:** Manage logins and permissions.
- **E-commerce:** Tracking orders and inventory.

TUTORIAL PREREQUISITES

Local Server

Ensure XAMPP, WAMP, or MAMP is installed and running Apache/MySQL.

PHP 8.x+

We recommend the latest stable version for performance and security.

Text Editor

VS Code, PHPStorm, or any IDE with syntax highlighting.

MYSQLI VS PDO

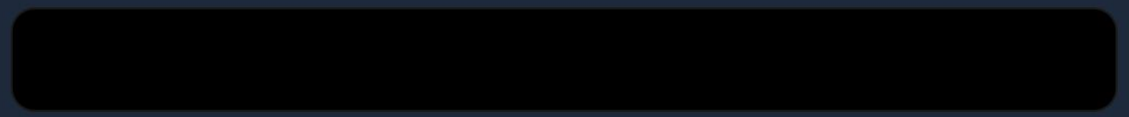
FEATURE	MYSQLI (IMPROVED)	PDO (PHP DATA OBJECTS)
Database Support	MySQL ONLY	12+ Different Databases
API Style	Procedural & OO	Object-Oriented ONLY
Security	Prepared Statements	Prepared Statements
Named Parameters	No	Yes (e.g., :username)

Pro Tip: Use PDO if you might switch databases later; Use MySQLi for MySQL-only performance.

Module 1

MySQLi Procedural Connection

THE PROCEDURAL WAY



Simplicity First

The `mysqli_connect()` function is the foundation. It requires four parameters to establish a tunnel to your data.

This method is highly favored by beginners due to its linear, step-by-step logic.

HANDLING CONNECTION ERRORS

- ⚠ **mysqli_connect_error():** Returns the error description from the last connection attempt.
- ✋ **die() Function:** Stops the script immediately. Crucial for preventing further execution on a broken link.
- 🚫 **Security Note:** Never show raw error messages to public users; log them to a file instead.

Module 2

MySQLi Object-Oriented

THE OBJECT-ORIENTED APPROACH

```
connect_error) {
    die("Error: " . $conn→connect_error);
}
echo "Connected via OO";
?>
```

Cleaner Syntax

Instead of passing the link variable to every function, you call methods on the connection object: `$conn->query()`.

Benefits: Better integration with modern PHP classes and easier to maintain in large scale apps.

Module 3

PHP Data Objects (PDO)

UNDERSTANDING THE DSN

PDO uses a **Data Source Name (DSN)** string. This is what makes it "Portable."

```
"mysql:host=$host;dbname=$db;charset=utf8mb4"
```

Prefix

Identifies the driver (mysql, pgsql, sqlite).

Host

Location of server (localhost or IP).

Charset

Essential for security (utf8mb4).

CONNECTING WITH PDO

```
setAttribute(PDO::ATTR_ERRMODE,
      PDO::ERRMODE_EXCEPTION);

  echo "Connected!";
} catch(PDOException $e) {
  echo "Connection failed: " .
      $e->getMessage();
}
?>
```

- **Exception Mode:** Forces PDO to throw errors as exceptions.
- **Try/Catch:** The professional way to handle failures.
- **Persistence:** Easily supports permanent connections.

PDO PERFORMANCE METRICS

0.05ms

Connection Overhead

Efficiency Matters

Modern PDO drivers are extremely thin layers over native MySQL drivers. The performance difference between MySQLi and PDO is negligible for 99% of applications.

Focus on **Code Quality** over micro-optimizations.

Critical Section

Security & SQL Injection

THE ENEMY: SQLI

SQL Injection happens when raw user input is concatenated directly into your SQL string.

```
 "SELECT * FROM users WHERE user = '" .
$_POST['u'] . "';"
```

An attacker can input `' OR 1=1 --` to bypass logins or `'; DROP TABLE users;` to delete data.

THE HERO: PREPARED STATEMENTS

Template

SQL is sent to the server with "placeholders" instead of data.

Bind

Data is sent separately. The DB engine treats it only as text, never executable code.

Execute

The DB runs the pre-compiled query with the provided data.

SECURE PDO EXECUTION

```php
// 1. Prepare Statement with Named Placeholders
$stmt = $pdo->prepare("INSERT INTO users (name, email) VALUES (:name, :email)");

// 2. Bind Values and Execute
$stmt->execute([
    'name' => $_POST['username'],
    'email' => $_POST['email']
]);

echo "User registered safely!";
```

Notice how `:name` is used instead of a direct variable. This ensures the input cannot "escape" the string context.

Module 4

The CRUD Lifecycle

DATABASE LIFECYCLE

CREATE

Defining schema and tables.

INSERT

Populating rows with data.

READ

Querying and fetching results.

UPDATE

Modifying existing records.

PDO FETCHING STRATEGIES

FETCH_ASSOC

Returns an array indexed by column name. `$row['id']`

FETCH_OBJ

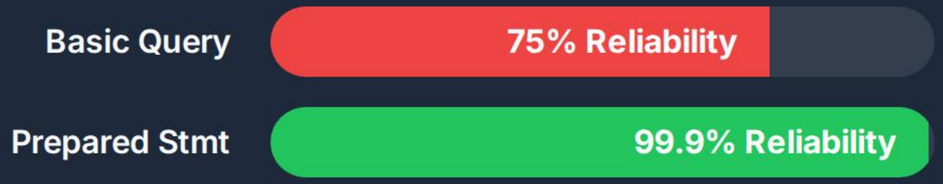
Returns an anonymous object. `$row->id`

FETCH_CLASS

Injects data directly into an existing custom class instance.

Default to **FETCH_ASSOC** for simple scripts.

EXECUTION STABILITY



Why use Prepared Stmts?

It's not just about security. It's about **Reliability**. Prepared statements handle special characters (like quotes) automatically, preventing syntax errors that crash your site.

PROJECT ARCHITECTURE

1. db_config.php

Centralize your connection details. Don't repeat yourself!

```
define('DB_HOST', 'localhost');
define('DB_USER', 'root');
define('DB_PASS', 'secret');

function getDB() {
  // Return PDO object
}
```

2. index.php

Include the config and perform your logic.

```
require 'db_config.php';
$db = getDB();
$users = $db->query("...");
```

FINAL BEST PRACTICES

- Always use **UTF-8** (utf8mb4) to support emojis and prevent corruption.
- Close connections implicitly by setting objects to `null`.
- Store passwords using `password_hash()`, not in the DB as plain text.
- Use environment variables (`.env` files) for production credentials.

COURSE SUMMARY

Connect

Use PDO for modern, portable apps.

Secure

Always use Prepared Statements.

Expertise

Handle errors gracefully with try/catch.

Q&A

Got Questions? Now is the time!

Further Reading: php.net/manual/en/book.pdo.php

Demo Code: github.com/tutorial-examples/php-db

IMAGE SOURCES

https://www.cloudways.com/blog/wp-content/uploads/Main-Image_750×394-146.jpg

Source: www.cloudways.com

https://static.vecteezy.com/system/resources/previews/057/960/169/non_2x/shield-icon-with-a-digital-lock-and-cybersecurity-concept-vector.jpg

Source: www.vecteezy.com

https://png.pngtree.com/png-vector/20250904/ourlarge/pngtree-green-checkmark-tick-icon-minimal-vector-isolated-high-resolution-design-for-png-image_17366295.webp

Source: pngtree.com

https://w0.peakpx.com/wallpaper/771/62/HD-wallpaper-php-programming-language-minified-html-java-code-syntax-highlighting-programming-pascal-assembler-css.jpg

Source: www.peakpx.com